
```
class Vehicle:
    def __init__(self):
        print('Vehicle created.')
    def startIgnition(self):
        pass # Ignition starting code goes here.
    def changeTire(self):
        pass # Tire changing code goes here.

class Car(Vehicle):
    def __init__(self):
        print('Car created.')

class Motorcycle(Vehicle):
    def __init__(self):
        print('Motorcycle created.')

class LunarRover(Vehicle):
    def __init__(self):
        print('LunarRover created.')
```

But all future changes to `Vehicle` will affect these subclasses as well. What happens if we need a `changeSparkPlug()` method? Cars and motorcycles have combustion engines with spark plugs, but lunar rovers don't. By favoring composition over inheritance, we can create separate `CombustionEngine` and `ElectricEngine` classes. Then we design the `Vehicle` class so it “has an” engine attribute, either a `CombustionEngine` or `ElectricEngine` object, with the appropriate methods:

```
class CombustionEngine:
    def __init__(self):
        print('Combustion engine created.')
    def changeSparkPlug(self):
        pass # Spark plug changing code goes here.

class ElectricEngine:
    def __init__(self):
        print('Electric engine created.')

class Vehicle:
    def __init__(self):
        print('Vehicle created.')
        self.engine = CombustionEngine() # Use this engine by default.

--snip--
```